

# Подготовка к экзамену по C++ (ИТМО)

## Тема 4-5: STL. Контейнеры, Итераторы и Алгоритмы

Hardcore Theory Edition

15 июня 2026 г.

### 1 Контейнеры STL: Кто есть кто?

STL предоставляет структуры данных на все случаи жизни. Главный вопрос экзамена: **"Какой контейнер выбрать и почему?"** Оценивается всегда по скорости вставки (в конец/в середину) и скорости поиска ( $O$ -большое).

#### 1.1 `std::vector` (Динамический массив)

**Хит по умолчанию.** Элементы лежат в памяти **сплошным куском** (один за другим).

- **Скорость:** Поиск по индексу `v[i]` —  $O(1)$ . Вставка в конец — амортизированное  $O(1)$  (иногда  $O(N)$  из-за реаллокации). Вставка в середину —  $O(N)$  (надо сдвигать элементы).
- **Кэш-френдли:** Поскольку данные лежат подряд, процессор считывает их огромными блоками в сверхбыстрый кэш. Поэтому вектор почти всегда быстрее списка, даже если вы часто вставляете элементы в середину!
- **Size vs Capacity:** `size` — сколько элементов сейчас есть. `capacity` — сколько памяти реально выделено. При переполнении вектор выделяет новый кусок памяти в 1.5 или 2 раза больше старого, копирует туда элементы и удаляет старый кусок (*Реаллокация*).

#### 1.2 `std::list` (Двусвязный список)

Каждый элемент лежит в случайном месте кучи и хранит два указателя: на следующий и предыдущий элементы.

- **Скорость:** Поиска по индексу нет (надо идти по цепочке,  $O(N)$ ). Вставка **в любое место** за  $O(1)$  (если уже знаешь, куда вставлять).
- **Кэш-враг:** Переход от узла к узлу заставляет процессор прыгать по всей памяти. Очень медленно.

### 1.3 `std::map` и `std::set` (Сбалансированные деревья)

Основаны на красно-черных деревьях. Элементы **всегда отсортированы**.

- **Скорость:** Поиск, вставка, удаление — строго  $O(\log N)$ .
- В `map` ключи уникальны. В `multimap` — могут повторяться.

### 1.4 `std::unordered_map` (Хэш-таблицы)

- **Скорость:** Поиск и вставка в среднем  $O(1)$  (быстрее, чем дерево!), но в худшем случае (коллизии хэшей) деградирует до  $O(N)$ . Элементы лежат **не упорядоченно**.

## 2 Итераторы (Мост между контейнером и алгоритмом)

Алгоритмам (например, `std::sort`) плевать, какой у вас контейнер. Им нужны только **итераторы**. Итератор — это умная обертка над указателем. Он перегружает оператор `++` (шаг вперед), `*` (взять значение) и `==` / `!=`.

#### Суть на пальцах (Жизненная аналогия)

Контейнеры — это разные виды полок (книжный шкаф, сейф, картотека). Алгоритм — это слепой библиотекарь, который умеет сортировать книги. Итератор — это собака-поводырь. Библиотекарь не знает, как устроен шкаф. Он просто говорит собаке: "Шаг вперед!"(`++it`) или "Дай книгу!"(`*it`), и собака сама разбирается, как достать элемент с конкретной полки.

### 2.1 Категории итераторов (Важно!)

Не каждый алгоритм работает с каждым итератором.

1. **Forward Iterators:** Могут идти только вперед (`std::forward_list`).
2. **Bidirectional Iterators:** Могут идти вперед и назад (`std::list`, `std::map`). Нельзя сделать `it + 5`.
3. **Random Access Iterators:** Могут прыгать на любую позицию за  $O(1)$ . Поддерживают `it + 5` и `it1 - it2`. (`std::vector`, массивы). *Например, `std::sort` требует именно Random Access итераторы, поэтому вы не можете отсортировать `std::list` с помощью `std::sort`!*

### 2.2 Инвалидация итераторов (Смертельная ловушка)

Если вы запомнили итератор на элемент `vector`, а потом сделали `push_back()`, вектор мог сделать реаллокацию (переехать в другой кусок памяти). Ваш старый итератор теперь указывает на "снесенный дом" (Dangling pointer). Использование такого итератора — **Undefined Behavior**.

**Правило:**

- У `vector` и `string` вставка/удаление инвалидирует итераторы.

- У `list`, `map`, `set` добавление элементов **никогда** не инвалидирует итераторы (узлы остаются на своих местах в памяти).